



INSTITUTO TECNOLÓGICO DE BUENOS AIRES

Cloud Provider Analytics

Primera Entrega - Diseño Preliminar

72.80 - Big Data

Primer Cuatrimestre - 2026

Integrantes:

Camila Lee	63382
Lucas Perri	62746

1. Caso de uso y justificación del enfoque Big Data

El proyecto plantea la construcción de una plataforma analítica para un proveedor de servicios en la nube. Los datos resultantes deben servir a tres áreas de la organización con preguntas distintas: el área financiera (control de costos y facturación a clientes), el área de soporte técnico (calidad del servicio percibido) y el área de producto (seguimiento del consumo de funcionalidades nuevas como las basadas en GenAI).

El desafío técnico tiene dos dimensiones que el enunciado destaca explícitamente. Por un lado, la naturaleza temporal mixta de los datos: parte del análisis requiere visibilidad operativa con latencia baja, mientras que otra parte se compone de cálculos consolidados con cadencia mensual. Por el otro, la calidad heterogénea del dato fuente: registros con campos faltantes, tipos no homogéneos, valores anómalos, y un cambio de esquema que ocurrió a mitad del histórico provisto.

La conjunción de estos factores justifica el encuadre del problema dentro del marco conceptual de Big Data, caracterizado por las V's vistas en clase. La siguiente tabla mapea cada V a un aspecto concreto del enunciado.

V	Manifestación en este caso de uso
Volumen	El feed de eventos de uso se entrega como archivos JSONL fragmentados que simulan la llegada en micro-lotes de un sistema productivo. Sumado a tres meses de facturación, encuestas NPS temporales, tickets, y los maestros de clientes, usuarios y recursos, el volumen agregado excede lo manejable eficientemente con bases relacionales tradicionales para procesamiento analítico.
Velocidad	El enunciado distingue dos regímenes de procesamiento. Las métricas de uso y costos incrementales requieren tratamiento near real-time; en contraste, los datos maestros y la facturación admiten procesamiento batch con cadencia diaria o mensual. La detección de anomalías de costo, en particular, debe ser oportuna para resultar accionable antes del cierre del período facturable.
Variedad	Conviven formatos estructurados (siete archivos CSV con esquemas diferenciados) y semi-estructurados (JSONL con anidamiento variable). Cada servicio (<i>compute</i> , <i>storage</i> , <i>genai</i> , etc.) tiene además sus propios campos específicos en los eventos.
Veracidad	El enunciado describe explícitamente la presencia de inconsistencias en los datos crudos: nulos en campos como NPS, CSAT y unit; tipos ambiguos como <i>value</i> llegando ocasionalmente como string; valores fuera de rango como <i>cost_usd_increment</i> negativo o con spikes atípicos. Sin una política explícita de calidad y aislamiento de datos malos, los <i>marts</i> derivados resultarían poco confiables.
Valor	El valor de la solución se materializa en las cinco consultas que el enunciado exige responder desde Cassandra: costos y requests diarios por organización y servicio, top-N servicios por costo acumulado, evolución de tickets críticos con tasa de SLA breach, revenue mensual normalizado a USD, y tokens GenAI con costo estimado. Cada tabla del serving layer debe corresponder a una de estas consultas.

2. Diagrama de Arquitectura de Alto Nivel

2.1 Patrón elegido: Lambda

La consigna requiere elegir explícitamente entre los patrones Lambda y Kappa, e implementar como mínimo el procesamiento streaming de eventos junto con el procesamiento batch de maestros y facturación. La arquitectura propuesta adopta el patrón **Lambda**, organizado sobre las tres zonas medallion del Data Lake (Bronze, Silver, Gold) que permiten progresión incremental de la calidad del dato.

Justificación de la elección. Los datos provistos por el enunciado tienen naturalezas distintas. Los archivos CSV de maestros, soporte, marketing, encuestas NPS y facturación representan estados que se actualizan con cadencia humana o cierre de período; no son streams de eventos. El feed *usage_events_stream*, en contraste, se entrega fragmentado precisamente para simular la dinámica de un sistema productivo de eventos en tiempo real. El patrón **Lambda** permite tratar cada flujo según su naturaleza, sin forzar transformaciones artificiales en ninguna dirección.

Por qué se descarta Kappa. Tratar todo como stream implicaría introducir un mecanismo de captura de cambios sobre los CSV maestros y la facturación, lo que sumaría complejidad operativa sin un beneficio analítico claro para este caso. Adicionalmente, la implementación canónica de Kappa requiere un broker durable y replayable (típicamente Apache Kafka) que no es parte del stack provisto por el enunciado. Forzar Kappa exigiría incorporar un componente fuera del alcance académico del proyecto.

2.2 Vista general de la arquitectura

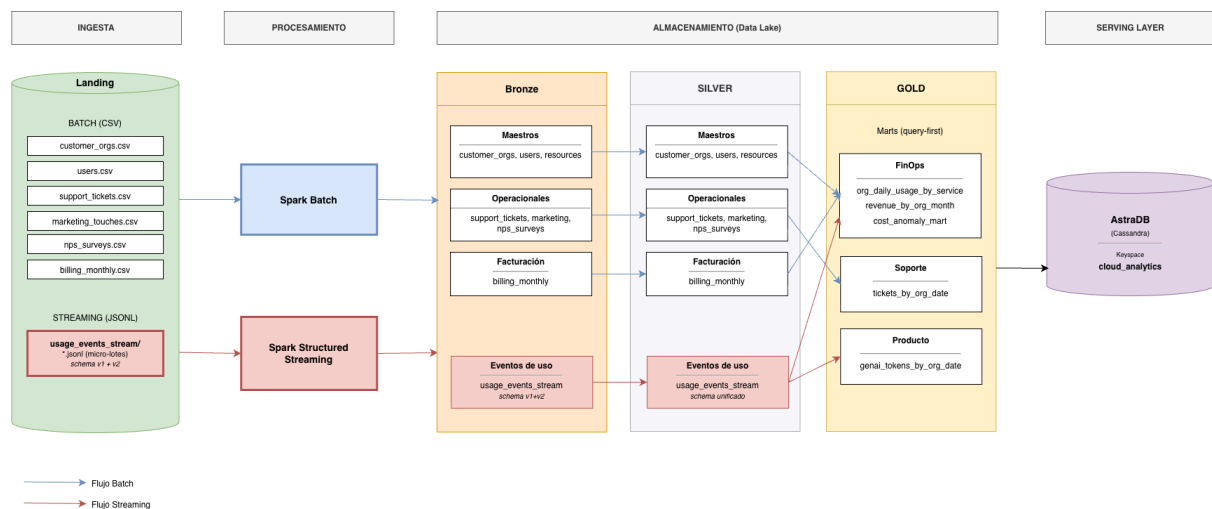


Figura 1: Arquitectura de alto nivel con los cinco componentes que pide la consigna (ingesta, procesamiento, almacenamiento, serving, flujos). Las operaciones aplicadas en cada zona se detallan en la sección 4.

El diagrama está organizado en las cuatro capas que pide la consigna. En *Landing* aterrizan inmutablemente los siete archivos CSV (procesamiento batch) y el directorio de archivos JSONL (procesamiento streaming). El procesamiento se realiza con dos motores paralelos: Spark Batch para los CSV y Spark Structured Streaming para el feed. El almacenamiento intermedio recorre las tres zonas medallion en formato Parquet, donde los datos se enriquecen progresivamente. El *serving layer*

carga los *marts* de Gold en AstraDB, donde quedan disponibles para las herramientas de visualización.

3. Mapeo de Requisitos a Componentes

La siguiente tabla relaciona los requisitos clave del enunciado con los componentes elegidos para resolverlos, las V's de Big Data involucradas, y una justificación corta de la elección.

Requisito	Componente	5V	Justificación
Procesamiento batch de maestros y facturación	Spark Batch (PySpark)	Volumen	Stack prescrito por el enunciado. Procesa los siete CSV's con cadencia diaria/mensual y los lleva a Parquet particionado.
Procesamiento en tiempo real	Spark Structured Streaming	Velocidad	Tecnología prescrita por el enunciado para el feed de eventos JSONL. Permite latencia near real-time con watermarks y dedup nativos.
Almacenamiento escalable de datos crudos	Landing + Bronze (Parquet)	Volumen, Variedad	Landing preserva los archivos originales. Bronze los materializa en formato analítico columnar y particionado.
Soporte a fuentes heterogéneas (CSV + JSONL)	Spark + esquema explícito	Variedad	La API DataFrame de Spark unifica el tratamiento de formatos estructurados y semi-estructurados con el mismo código.
Manejo de schema evolution v1↔v2	Esquema unión nullable	Variabilidad	Los campos <i>carbon_kg</i> y <i>genai_tokens</i> se declaran nullable. Filas v1 quedan con NULL, filas v2 pobladas, sin código condicional.
Calidad de datos y filas inválidas	Reglas en Silver + quarantine	Veracidad	Filas que violan reglas (<i>event_id</i> nulo, valores fuera de rango) se desvían a Parquet aparte para inspección y reproceso.
Detección de anomalías de costo	z-score + MAD + p99 en Silver	Veracidad, Valor	Tres métodos en paralelo, flag activo cuando coinciden al menos dos. Reduce falsos positivos respecto a un único método.
Idempotencia y reprocesabilidad	Checkpointing + claves naturales + UPSERTs	Veracidad	Requisito explícito del enunciado. Permite re-ejecutar jobs sin duplicar datos en las zonas posteriores.
Particionamiento eficiente	Parquet particionado por fecha y servicio	Volumen, Velocidad	Requisito explícito del enunciado. Las consultas que filtran por fecha o servicio leen sólo los archivos relevantes.
Serving para dashboards de baja latencia	AstraDB (Cassandra)	Velocidad, Valor	Tecnología prescrita por el enunciado. Modelado query-first: una tabla por cada consulta crítica del negocio.

4. Flujo de Datos (Data Pipeline)

El pipeline contempla los dos modos de procesamiento exigidos por el enunciado: batch para los datos con cadencia humana, streaming para el feed de eventos. Ambos modos recorren las cuatro zonas del Data Lake y confluyen en AstraDB. La siguiente figura ilustra las operaciones aplicadas en cada zona.

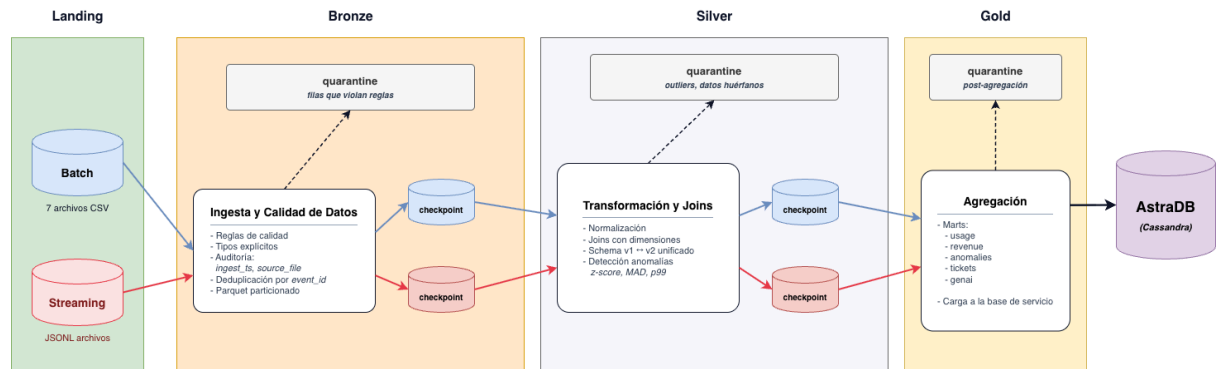


Figura 2: Operaciones aplicadas en cada zona del Data Lake. La zona quarantine en Bronze y Silver aísla las filas que no pasan las reglas; los checkpoints permiten reanudar el stream tras fallos.

4.1 Pipeline Batch (jobs diarios y mensuales)

Paso	Herramienta	Qué hace	Zona destino
1. Lectura	PySpark	Lee los siete archivos CSV de Landing con esquema explícito.	Landing → Bronze
2. Estandarización	PySpark	Aplica tipos, agrega <code>ingest_ts</code> y <code>source_file</code> , escribe Parquet particionado.	Bronze
3. Limpieza	PySpark	Aplica reglas de calidad y <code>joins</code> con dimensiones; filas inválidas a quarantine.	Silver
4. Agregación	PySpark	Calcula los <code>marts</code> pre-agregados de FinOps, Soporte y GenAI.	Gold
5. Carga	PySpark	Escribe cada <code>mart</code> Gold a su tabla Cassandra mediante UPSERT.	AstraDB

4.1 Pipeline Streaming

Paso	Herramienta	Qué hace	Zona destino
1. Lectura	Spark Streaming	Lee los archivos JSONL del directorio.	Landing → Bronze
2. Limpieza	Spark Streaming	Acota datos atrasados y deduplica por <code>event_id</code> .	Bronze

3. Transformaciones	Spark Streaming	Joins, conformación de esquema, flags de anomalía.	Silver
4. Agregación	Spark Streaming	Calcula los marts agregados por ventana de tiempo.	Gold
5. Escritura	Spark Streaming	Escribe los datos consolidados en Cassandra.	AstraDB

5. Asunciones y Riesgos Iniciales

5.1 Asunciones

ID	Asunción	Mitigación
A1	Volumen del dataset compatible con Colab gratuito (orden de < 1 GB por archivo).	Si excede el runtime, el mismo código Spark se puede correr en infraestructura más grande sin reescribir la lógica.
A2	Los archivos JSONL llegan al directorio cada algunos minutos, no a tasa de segundos.	El intervalo de procesamiento se ajustará empíricamente cuando trabajemos con los archivos reales.
A3	El FX para revenue se obtiene del propio archivo <i>billing_monthly</i> o se asume USD por defecto.	Si surge ambigüedad sobre la fuente del FX, se definirá una tabla simple de tasas mensuales.
A4	El esquema permanece en versiones v1 y v2 durante todo el proyecto.	Si apareciera una v3, se extiende el esquema con los campos nuevos como opcionales.
A5	El free tier de AstraDB alcanza para los cinco <i>marts</i> del proyecto.	Si los marts crecen más de lo previsto, se reduce el volumen guardando solo lo necesario.

5.2 Riesgos y mitigaciones

ID	Riesgo	Mitigación
R1	Latencia en streaming sin broker dedicado.	Procesar los archivos en lotes pequeños y dar un margen amplio para datos atrasados.
R2	El cambio de esquema (v1 $\rightarrow$ v2) puede romper la lectura si no se contemplan todos los campos nuevos.	Definir desde el inicio un esquema que incluya los campos de ambas versiones, validando con muestras.
R3	Las consultas no anticipadas pueden no calzar con el modelado de Cassandra.	Validar el modelado contra las cinco consultas que pide el enunciado antes de cargar datos.
R4	Los algoritmos de detección de anomalías pueden generar falsos positivos.	Combinar los tres métodos que pide la consigna y marcar como anómalo solo cuando coinciden al menos dos.
R5	Colab puede desconectarse y perder el estado durante un job largo.	Guardar el progreso en Drive periódicamente para poder retomar desde el último punto.

6. Estimación de Esfuerzo y Recursos

6.2 Equipo (2 personas) y roles

Rol principal	Responsabilidades
Data Engineer	Ingesta batch y streaming, transformaciones Bronze→Silver, calidad y anomalías.
Data Modeler & Cassandra	Marts Gold, modelado query-first, keyspace y carga a AstraDB, consultas CQL.
Compartido	Documentación, presentación, video, integración end-to-end.

6.2 Cronograma estimado

La planificación se ancla en los tres hitos oficiales del curso: la entrega de diseño preliminar (hoy, 04/05), el MVP técnico del Segundo Parcial (15/06) y la entrega final (06/07 o 13/07). El alcance del MVP está definido por la consigna del Segundo Parcial: end-to-end mínimo con Bronze, Silver mínimo, un mart FinOps en Gold y dos consultas en Cassandra.

Fase	Semanas	Entregable
1. Setup y diseño preliminar	04/05 – 18/05	Este documento. Provisión de repo, AstraDB y notebook base.
2. Bronze: ingesta batch + stream	18/05 – 01/06	Ingesta de 3 maestros + stream JSONL a Bronze, con checkpoint.
3. Silver mínimo + Gold + 1 tabla Cassandra	01/06 – 15/06	Reglas de calidad, quarantine, mart FinOps, 2 consultas CQL. Hito MVP.
4. Gold completo + resto de marts	15/06 – 29/06	Marts de Soporte y Producto/GenAI completos, las 5 consultas mínimas del enunciado.
5. Documentación y video	29/06 – 06/07 o 13/07	Presentación, video explicativo, repo limpio. Hito final.